

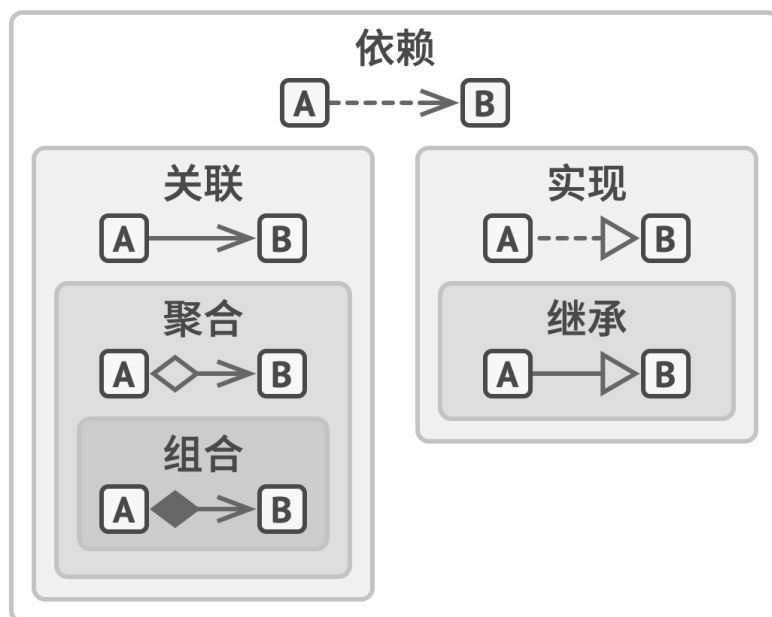
软件系统设计复习

- 软件系统设计复习

- Part I

- 对象间关系

-



对象和类之间的关系：从弱到强。

- 依赖 dependency

- 最基础、最微弱的关系：修改一个类的定义可能导致另一个类的变化
具体而言：代码中使用具体类的名称（使用接口或抽象类降低依赖程度）

- 关联 association

- 一个对象使用另一对象或者与另一对象交互
具体而言：一个对象总是拥有访问与其交互的对象的权限，即类的字段

- 聚合 polymerization

- 一个对象拥有一组其他对象，扮演容器或集合的角色，组件可以独立于容器存在

- 组合 composition

- 一个对象由一个或多个其他对象实例构成，组件不能独立于容器存在

- 实现 implement

- 实现公共接口

- 继承 inherit

- 代码复用

- 软件模式

- 将模式的一般概念应用于软件开发领域，即软件开发的总体指导思路或参照样板。软件生存期的每一个阶段都存在一些被认同的模式。

它是对软件开发这一特定问题的解法的某种统一表示，软件模式等于一定条件下出现的问题及其解法。包含：问题描述、前提条件（环境或约束条件）、解法和效果。

软件模式与具体应用领域无关，模式发现需要遵循大三律，即需要经过三个以上不同类型/领域的系统检验才能从候选模式升格为模式。

- 设计模式

设计模式是一套被反复使用、多数人知晓的、经过分类编目的代码设计经验的总结，包含：模式名称、问题、解决方案、效果

- 主要类别

- 按目的划分

- 创建型模式

- 提供创建对象的机制，增加已有代码的灵活性和可复用性

- 结构型模式

- 将对象和类组装成较大的结构，并同时保持结构的灵活和高效

- 行为模式

- 负责对象间的高效沟通和职责委派

- 按范围划分

- 类模式

- 处理类和子类之间的关系，编译时静态确定，包括工厂模式、类适配器模式和模板方法模式

- 对象模式

- 处理对象间的关系，在运行时动态确定

- 设计目标

- 代码复用（类库、模式【中间层】、框架），扩展性

- 设计原则

- 具体例子：电动汽车是汽车（继承），汽车有汽油引擎和电动引擎（组合，将行为委派给其他对象，而不是自行实现）

- 可变性封装原则：区分不变和变化的内容，最小化变更造成的影响

- The Least Knowledge Principle 最小知识原则：Law of Demeter 迪米特法则

- Composite Reuse Principle 合成复用原则（组合优于继承）：

- 子类不能减少父类的接口，即使它们没什么用

- 重写方法时需要确保新行为与基类中的版本兼容

- 继承打破了父类的封装

- 子类与父类紧密耦合

- 简单通过继承复用代码可能导致平行继承（两个维度以上的继承会导致类层次结构的膨胀）

- SOLID原则

- 不要过度使用某些原则，否则可能导致代码复杂度过高

原则之间的关系：

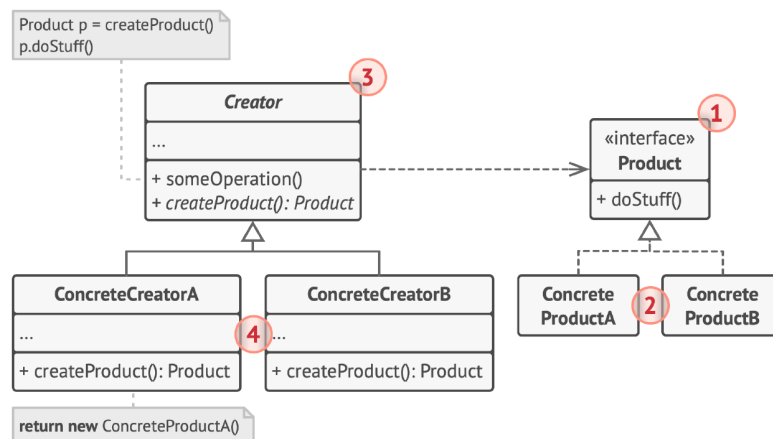
目标：开闭原则

指导：最小知识原则

基础：单一职责原则、可变性封装原则

实现：依赖倒转、合成复用、里氏替换、接口隔离

- Single Responsibility Principle 单一职责原则：修改类的原因只有一个
 - Open/Closed Principle 开闭原则：对扩展开放，对修改封闭
 - Liskov Substitution Principle 里氏替换原则：扩展一个类时，要在**不修改客户端代码**的情况下将子类的对象作为父类对象进行传递
 - 子类方法的参数类型必须与父类的参数类型相匹配或更抽象
 - 子类方法的返回值类型必须与父类返回值类型相匹配或更具体
 - 子类不应该抛出基础方法预期之外的异常
 - 子类不加强前置条件
 - 子类不削弱后置条件
 - 保留父类不变量（引入新的成员，不要直接修改）
 - 子类不能修改父类中私有成员的值
 - Interface Segregation Principle 接口隔离原则：客户端不应被强迫依赖于其不使用的方法
 - Dependency Inversion Principle 依赖倒置原则：高层次（业务逻辑）不应依赖低层次类（基础操作），而是都应该依赖接口；具体实现依赖于接口
- 创建型模式
 - 工厂方法 Factory Method



> 相比简单工厂的静态方法，核心的工厂类不再负责所有产品的创建，而是将具体的创建延迟到子类，从而做到了完全的开闭原则

>

> 常与反射机制结合使用

使用特殊的工厂方法替代直接通过 `new` 调用构造函数

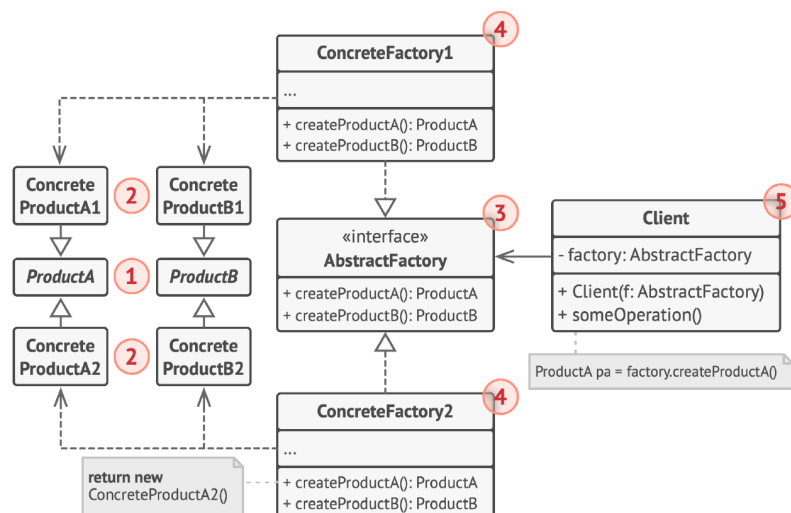
适用场景：

- 编写代码时无法预知对象确切类别及其依赖关系（创建与实际使用分离）
- 希望用户能扩展软件库或框架的内部组件
- 复用现有对象，而不是总是重新创建对象（`createProduct` 从对象池中搜索对象）

优缺点：

- 避免创建者和具体产品之间的紧密耦合
- 单一职责原则
- 开闭原则（客户端代码）
- 代码结构变得更加复杂

• 抽象工厂 Abstract Factory



增加新的具体工厂和产品族符合开闭原则，而增加新的产品（等级结构）则需要对接口进行扩展，不符合开闭原则（开闭原则的倾斜性）

每一种产品衍生出了许多变体，称同种产品的所有变体组成了产品系列，同一系列的所有产品组成一个产品族。客户端在调用得到的工厂对象时，得到的一定来自同一产品族（即属于同一系列），而客户端选择系列类型则是通过配置文件或运行环境设定完成的。

适用场景：

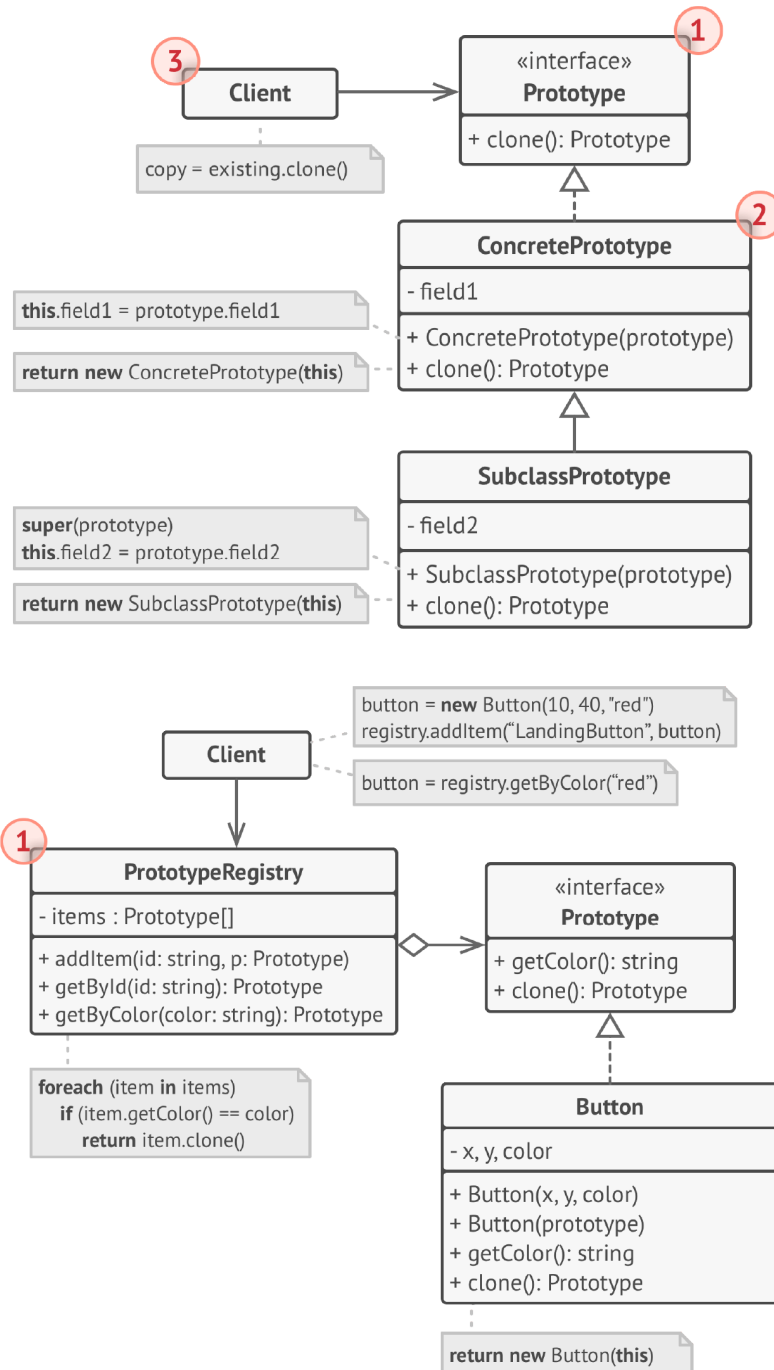
- 无法提前获取交互对象的产品系列类型，或不希望代码基于产品的具体类进行构建
- 一个类基于一组抽象方法，因此与多种类型产品发生了交互（违背了单一职责）

优缺点：

- 确保同一工厂生成的产品相互匹配

- 避免客户端与具体产品代码耦合
- 单一职责
- 开闭原则（客户端代码）
- 代码更加复杂

• 原型 Prototype



深克隆（hard）和浅克隆，改造已有类时违背开闭原则

复制已有对象，且无需使代码依赖它们所属的类（编码不需要知道它们属于哪一个类，也无需在意其中的私有成员）

实现方式是让需要被克隆的类实现 `clone` 方法，它可以访问自己内部的私有成员

运作方式是先创建一系列不同类型的对象，并完成对它们的配置，如果所需对象与预先配置的对象相同，则克隆原型

原型注册表：支持客户端方便地添加原型，查找并完成复制

适用场景：

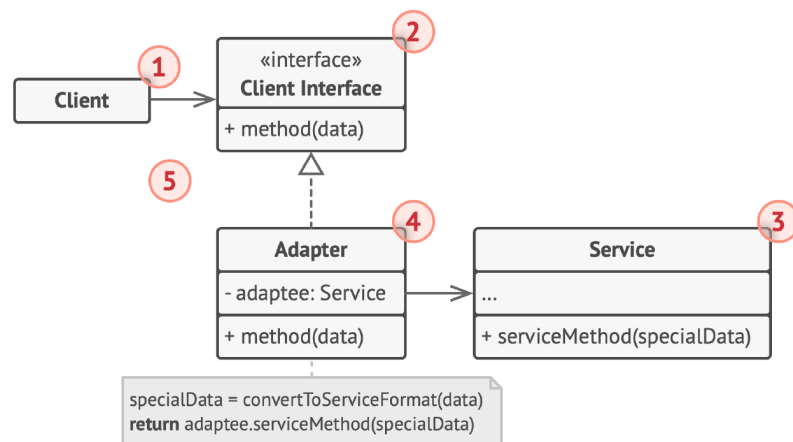
- 需要复制一些对象，但又希望代码独立于这些对象所属的具体类（即不直接读写这些具体类）
- 子类的区别仅在于对象的初始化方式，客户端调用的目的只是为了创建特定类型的对象

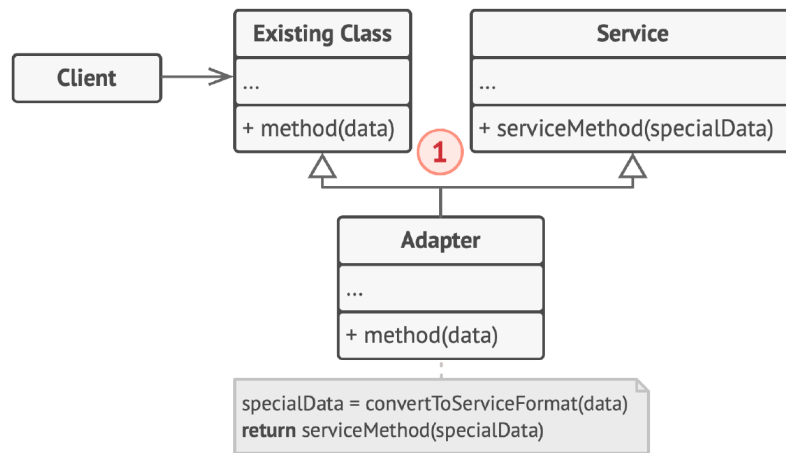
优缺点：

- 避免与具体类的耦合
- 避免反复运行初始化代码
- 更方便地生成复杂对象
- 处理复杂对象的不同配置的同时避免继承
- 克隆包含循环引用的复杂对象可能非常麻烦

• 结构型模式

• 适配器模式 Adapter





> 完全的开闭原则

>

> 类适配器的灵活性：适配器是Service/Adaptee子类，可以置换实际调用的接口方法

>

> 对象适配器的灵活性：可将所有Service/Adaptee的子类都适配到同一接口，但置换方法不容易，需要额外添加Service子类

>

> 默认/缺省适配器模式：抽象类实现部分接口（其他接口不需要）

>

> 双向适配器：适配器同时包含目标类和适配者类的引用，两者可以通过适配器互相调用

使接口不兼容的对象能够相互合作

对象适配器：

客户端只需要通过接口与适配器交互，无需与具体的适配器类耦合

类适配器：

通过多继承机制，类适配器无需封装任何对象，即可完成转换与目标服务的调用

适用场景：

- 希望使用的类的接口与其他代码不兼容

- 需要复用的类处于同一继承体系，它们都具有额外的一些共同方法，但这些方法不是继承体系中子类所具有的共性

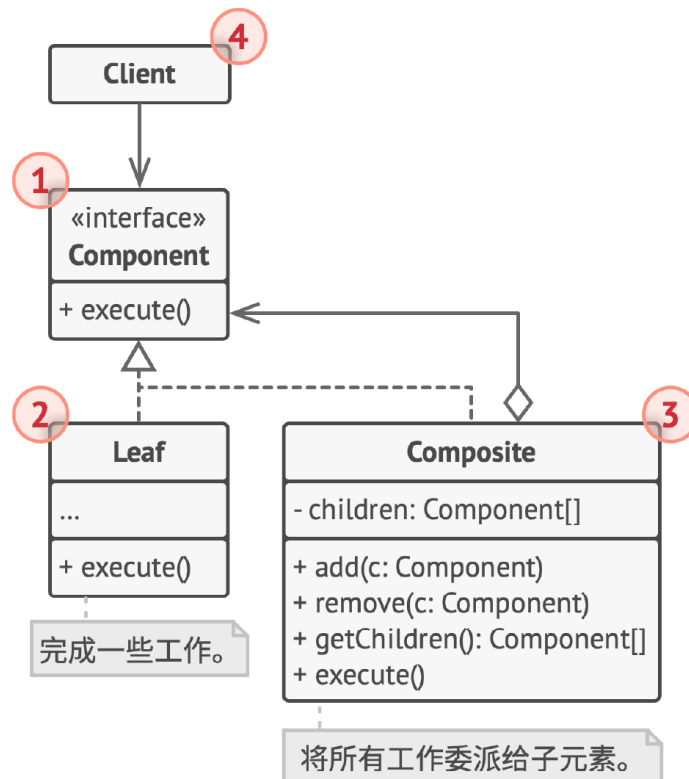
优缺点：

- 单一职责：分离接口或数据转化代码与主要业务逻辑代码

- 开闭原则：添加新类型适配器即可，无需修改客户端代码

- 代码复杂度增加

- 组合模式 Composite



> 容易添加新对象构件，但难以限制其类型

>

> 透明组合模式：抽象构件包含了所有定义

>

> 安全组合模式：抽象构件仅包含基础定义（如图）

将对象组合成树状结构，并像使用独立对象一样使用它们

标准的组合模式中，非叶子节点不存储额外数据，只负责控制、管理相关的工作

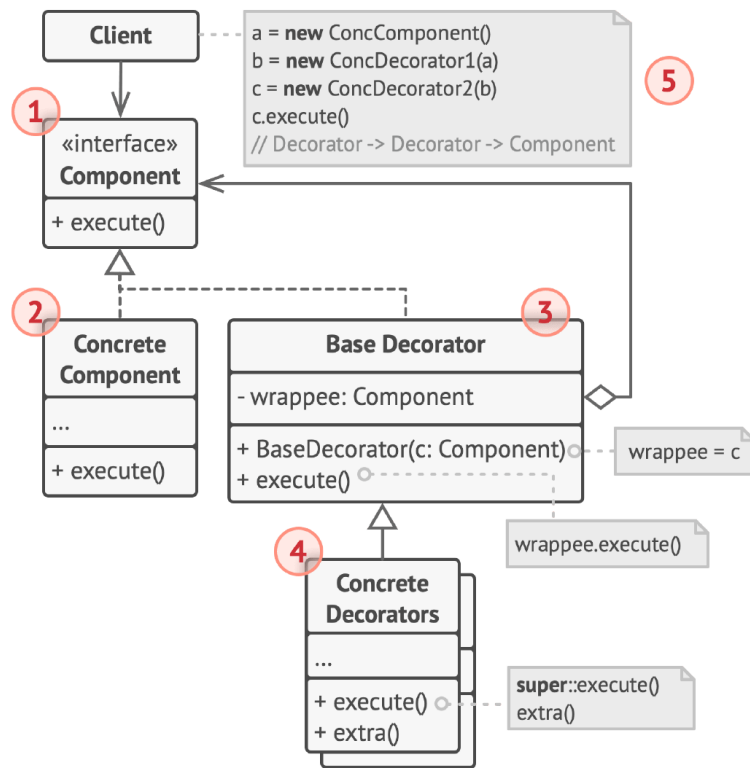
适用场景：

- 树状对象结构
- 客户端以相同方式处理简单和复杂元素

优缺点：

- 利用多态和递归更方便地使用复杂树结构
- 开闭原则：添加新元素即可使其成为对象树的一部分
- 功能差异较大的类难以提供容易被理解的公共接口

- 装饰模式 Decorator



> 使用关联动态地添加功能，相比静态的继承创建更多对象

>

> 简化：

>

> - 具体构件类轻量化

>

> - 装饰类接口与被装饰类接口保持一致

>

> - 如果只有一个具体构件，没有抽象构件，抽象装饰类可以作为具体构建类的直接子类

>

> 透明装饰模式（安全性要求高）：客户端完全针对抽象编程，不应声明为具体类

>

> 半透明装饰模式：允许声明具体装饰者类，调用具体装饰者中新增方法，但不能声明具体构件

通过将对象放入包含行为的特殊封装对象中来为原对象中，为原对象绑定新行为

核心思想是“组合优于继承”，采用包含与目标对象相同的一系列方法的封装器，封装器在将请求委派给目标对象前后对其进行处理。客户端将所有需要的封装器放入一系列需要的装饰中，形成栈结构

Concrete Component 对象作为基础，不断在其上包装需要的**Concrete Decorators**，最后调用最外围包装对象的`execute`，调用时先处理被包装对象的`execute`内容，最后调用自己的`extra`内容（可以调换顺序）

适用场景：

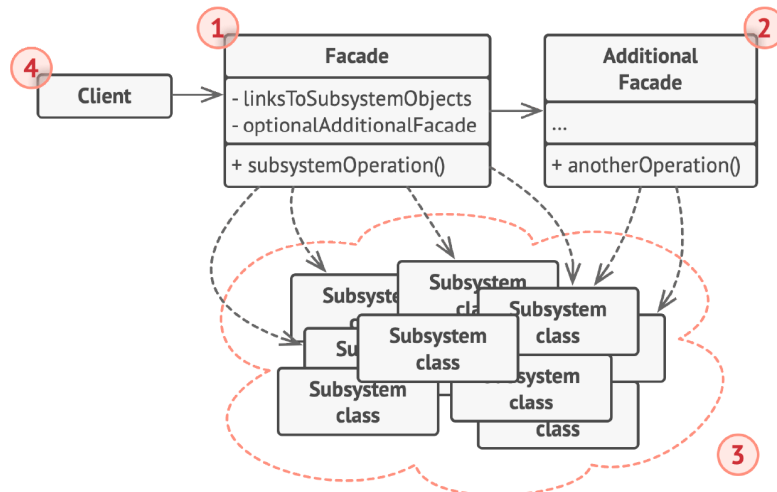
- 希望在无需修改代码的情况下即可使用对象，且希望在运行时为对象新增额外的行为

- 使用继承来扩展对象行为的方案难以实现或根本不可行

优缺点：

- 无需创建新子类即可扩展对象行为（在已有行为上组合）
- 可在运行时添加或删除对象的功能（根据配置文件组装不同的装饰堆栈）
- 单一职责：实现许多行为的大类拆分为多个小类
- 在装饰堆栈上删除特定封装器比较困难
- 实现行为不受装饰栈顺序影响的装饰比较困难
- 各层的初始化配置代码不优雅

• 外观模式 Facade



> 如果不引入抽象外观（结合工厂），增加新的子系统违背开闭原则

>

> 迪米特法则的实践

>

> 不能通过继承外观类为子系统添加新行为，而是应该更改子系统（更改不应浮于表面）

外观模式为程序库、框架或其他复杂类提供一个简单的接口，即如果内部的初始化依赖关系非常复杂，可以通过外观模式对外提供一个简单的接口，尽管功能受限，却包含了客户端真正关心的功能

附加外观可以避免多种不相关的功能污染单一外观，使其复杂化

适用场景：

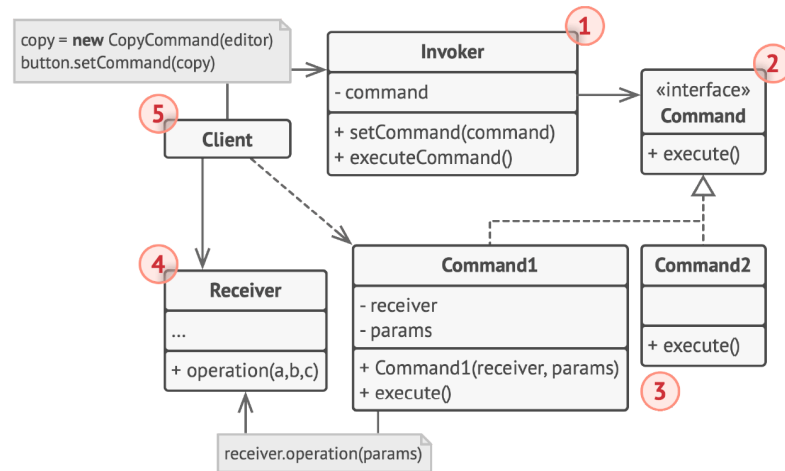
- 需要一个指向复杂子系统的直接接口，且该接口的功能有限
- 需要将子系统组织为多层结构（用不同的外观定义各层次的入口）

优缺点：

- 代码独立于复杂子系统
- 外观可能成为与程序中所有类都耦合的上帝对象

● 行为模式

● 命令模式 Command



> 封装命令，一个操作对应一个命令 --> 过多具体命令类

>

> 完全的开闭原则

将请求转换为一个包含与请求相关的所有信息的独立对象。支持方法参数化、延迟请求执行或放入队列，且实现可撤销操作

执行流程：

- 调用者 **Invoker** 对请求进行初始化，触发 **command** 成员的 **execute** 接口，该成员并不由 **Invoker** 创建，而是构造 **Invoker** 对象时通过客户端传入的参数
- 具体命令将调用参数信息 **params**（通过某种方式获取）委派给业务逻辑对象 **receiver**，为简化代码考虑，可以和该对象合并
- 接收者 **Receiver** 实现业务逻辑

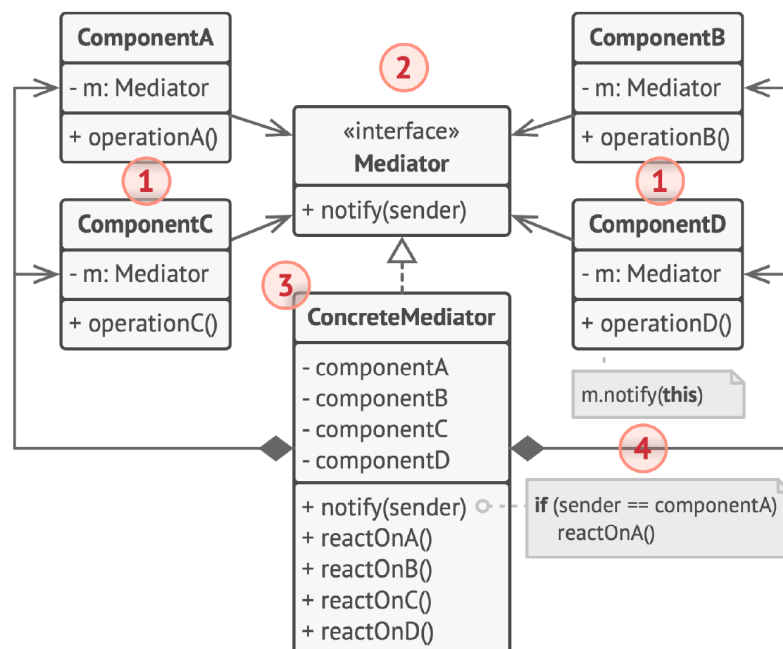
适用场景：

- 需要通过操作来参数化对象
- 需要将操作放入队列中、操作的执行或者远程执行操作
- 需要实现回滚功能（记录命令，反向执行）

优缺点：

- 单一职责：解耦触发和执行操作的类
- 开闭原则：创建新命令
- 撤销和恢复
- 操作的延迟执行
- 一组简单命令可以组合成一个复杂命令（ak. 宏命令，调用每一个子命令的execute）
- 代码更加复杂

• 中介者模式 Mediator



> 引用中转的结构性+协调合作的行为性

>

> 迪米特法则的典型应用（完全图转菊花图）

>

> 引入新组件时需要修改具体中介者类的代码

用于减少对象之间混乱无序的依赖关系，会限制对象之间的直接交互，迫使它们和一个中介者进行合作

类所拥有的依赖关系越少，就越易于修改、扩展或复用（迪米特法则）

运行方式：

- 组件 Component 是各种包含业务逻辑的类，包含一个指向中介者的引用

- 中介者 Mediator

接口声明了组件交流的方法，通常只包含一个通知方法，可将任意上下文作为该方法的参数

- 具体中介者 Concrete Mediator

封装了多种组件间的关系，通常会持有所有组件的引用并对其进行管理

- 组件间不互通，发生事件通知中介者

适用场景：

- 一些对象和其他对象紧密耦合以致难以对其进行修改

- 组件因过于依赖其他组件而无法在不同应用中复用

- 为了在不同情景下复用一些基本行为，导致需要被迫创建大量组件子类

优缺点：

- 单一职责：将组件间的通信交流抽取到同一位置

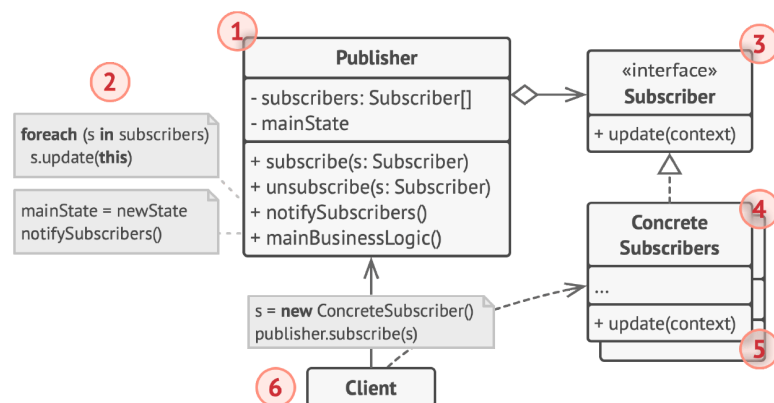
- 开闭原则：新增新的中介者

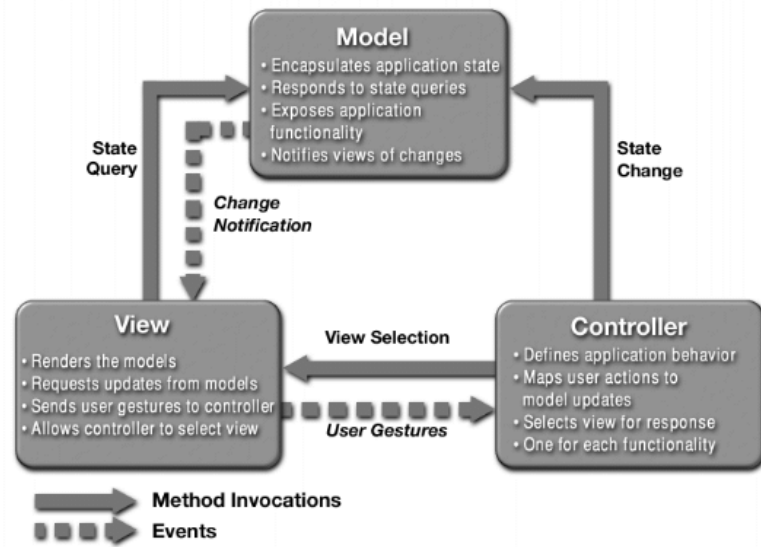
- 减轻耦合

- 方便复用

- 中介者可能成为上帝对象

- 观察者模式 Observer





> 松耦合👍：双方可以交互，但不太清楚彼此细节

>

> 完全的开闭原则

>

> ⚠️ 循环依赖可能导致系统崩溃

>

> 通知方式：

>

> - publisher push: 发生改变就通知，连续操作时效率低

>

> - subscriber pull:

> 需要时调用通知，避免中间更新，但容易出错（何时需要调用）

定义一种订阅机制，可在对象事件发生时通知多个“观察”该对象的其他对象

运行方式：

- 发布者 Publisher 向其他对象发送值得关注的事件
- 新事件发生时，发布者遍历订阅列表并调用每个订阅者对象的通知方法
- 订阅者 Subscriber 接口声明通知接口，通常仅包含 update 方法
- 具体订阅者执行操作回应通知

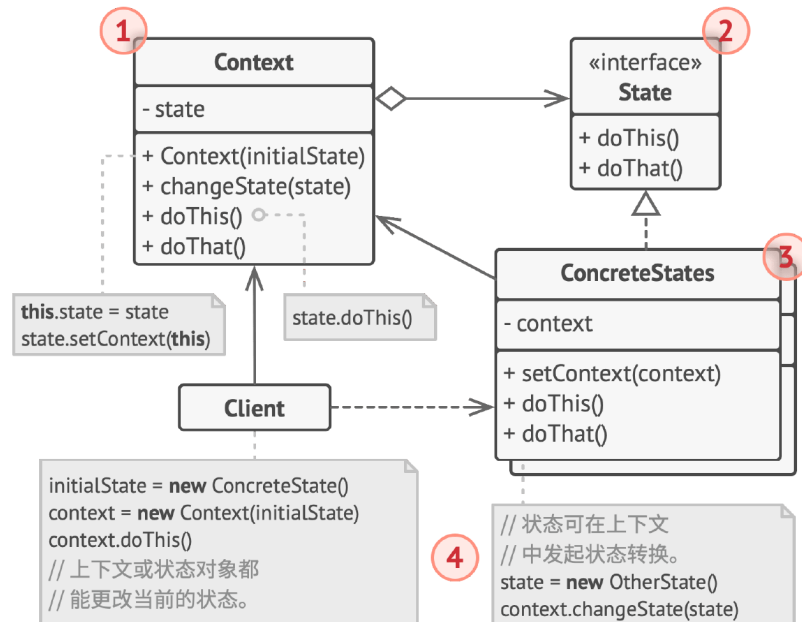
适用场景：

- 一个对象状态的改变需要改变其他对象，或实际对象是事先未知或动态变化时
- 一些对象必须观察其他对象时

优缺点：

- 开闭原则：根据接口类型引入对应的具体类
- 运行时建立对象之间的联系
- 订阅者的通知顺序是随机的

● 状态模式 State



- > 对开闭原则的支持不太好，添加新的状态类需要修改上下文类的代码；修改某个状态类的行为也需要修改代码
- >
- > 扩展：上下文类中状态成员的静态化
- >
- > 简单状态模式：无需完成状态切换（类似策略模式），开闭原则支持++

对象内部状态变化时改变其行为，看上去像改变了自身所属的类

运行过程：

- 上下文 Context 保存具体状态对象的引用，并负责状态相关的工作
- 状态接口声明各种状态的方法，为了避免多个状态中包含相似代码，可以封装中间抽象类或结合适配器模式
- 具体状态自行实现各状态的方法，并存储上下文对象的反向引用，以获取数据，触发状态转移
- 上下文和具体状态都可以设置上下文的下个状态

适用场景：

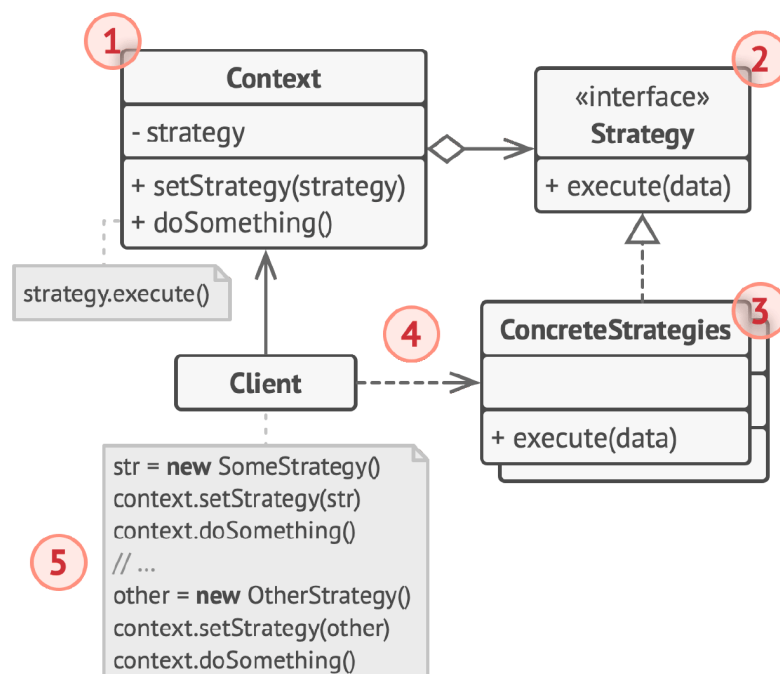
- 对象需要根据当前状态进行不同行为，同时状态数量非常多且与状态相关的代码会频繁变更

- 某个类需要根据成员变量的值改变行为，导致大量条件语句时
- 相似状态和基于条件的状态机转换中存在许多重复代码

优缺点：

- 单一职责
- 开闭原则：新状态的引入
- 消除臃肿的状态机条件语句
- 容易小题大做

● 策略模式 Strategy



定义一系列算法，并将每种算法放入独立的类中，使算法对象能够相互替换

适用场景：

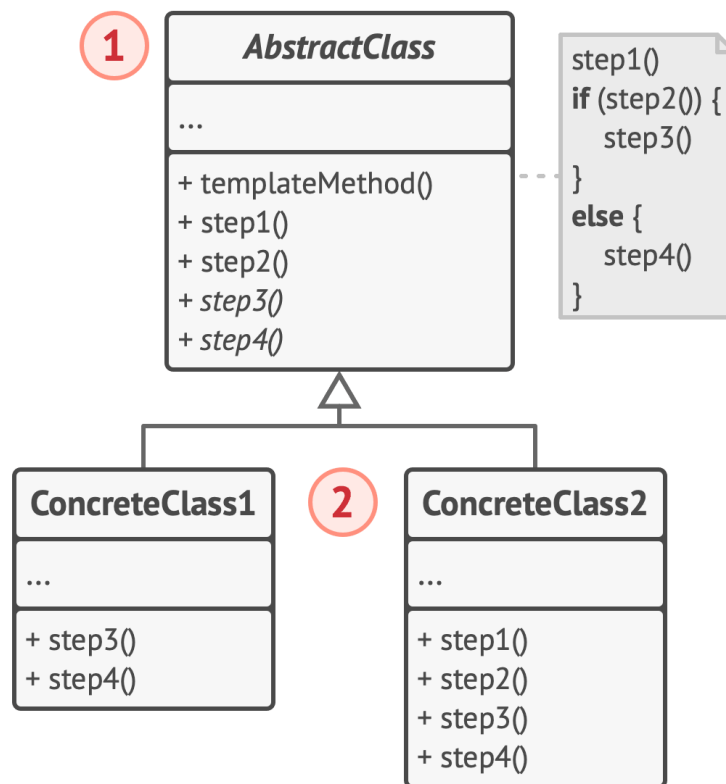
- 需要使用不同算法变体，并在运行时切换算法
- 有许多仅在执行某些行为时略有不同的相似类
- 算法在上下文的逻辑中不是特别重要
- 使用复杂条件运算符以在同一算法的不同变体中切换

优缺点：

- 运行时切换算法

- 分离算法的实现和使用
- 组合代替继承
- 开闭原则：新策略引入
- 算法改变少引起不必要的复杂化
- 客户端需要知晓策略间的不同
- 匿名函数更加简洁

● 模板方法模式 Template Method



> 基本方法：各个步骤，包含：

>

> - 抽象方法

>

> - 具体方法

>

> - 钩子方法（下图中的step2，判断是否执行分支中的基本方法，父类中常常实现空方法或默认实现，以执行默认流程，实现子类反向控制父类）

>

> 通过添加子类的扩展行为添加新的行为，完全的开闭原则

>

> 符合单一职责原则，类的内聚性高

>

> 继承的优秀实践

>

> 好莱坞原则：Don't call us, we'll call you，父类控制子类的整体逻辑流程

父类中定义算法框架，允许子类在不修改结构的情况下重写算法的特定步骤

适用场景：

- 只希望客户端扩展某个特定算法步骤，而非整体结构
- 多个类的算法除细微之处没有不同

优缺点：

- 仅可允许客户端重写大型算法中的特定部分，使得算法其他部分修改对其造成的影响减小
- 将重复代码提取到超类中
- 部分客户端受到算法框架的限制
- 通过子类抑制默认步骤实现可能违反里氏替换原则
- 模板方法步骤越多，其维护工作就可能会越困难

● 模式间的关系

- **工厂方法** vs 其他创建型模式：设计初期通常适用工厂方法，后续演化为其他创建型模式
- **抽象工厂**模式通常是一组**工厂方法**，但其中的产品也可以用原型模式来实现
- **原型**模式不基于继承，因此没有继承的缺点，但需要初始化
- **工厂方法**是**模板方法**的一种特殊形式，也可以作为模板方法中的一个步骤
- 当只需隐藏子系统创建对象的方式时，可以使用**抽象工厂**代替**外观**模式
- **原型**模式可以用于保存**命令**模式的历史记录
- 大量使用**组合**和**装饰**的设计通常可以从**原型**中获益，避免从零构造复杂结构
- **适配器**模式可以对已有对象的接口进行修改，**装饰**模式则是在不改变对象接口的前提下强化对象功能；此外，**适配器**无法实现递归
- **外观**为现有对象定义了一个新接口，**适配器**则试图运用已有接口，后者通常只封装一个对象，前者通常作用于整个对象子系统
- **状态**、**策略**、**组合**模式的核心思想都是将工作委派给其他对象
- **组合**和**装饰**都是依赖递归组织无限数量的对象，但纯粹的**组合**只对子节点“求和”
- **外观**模式和**中介者**模式的职责都是尝试在大量紧密耦合的类中组织起合作
- **命令**模式在接收者和请求者之间建立单向连接；**中介者**清除了两者的直接连接，强制进行间接沟通；**观察者**运行接收者动态地订阅或取消接收请求
- **命令**和**策略**模式都能通过某些行为参数化对象

- **中介者 vs 观察者**：中介者的主要目标是消除一系列系统组件之间的相互依赖；观察者的目标是在对象之间建立动态的单向连接（依赖仍然存在）
- **状态模式**可视为**策略模式**的扩展，它们都基于**组合机制**，但**策略**对象之间完全独立，而**状态**对象之间的依赖则不受限制
- **模板方法**基于继承机制，运作在类层次上，是静态的；**策略**基于组合机制，运作在对象层次上，是动态的

- Part II 软件架构

- 定义

- 结构 structure
 - 元素 elements
 - 关系 relationships
 - 设计 design

- 架构师的职责

- 联络模块
 - 软件工程的最佳实践
 - 技术知识
 - 风险管理

- 来源

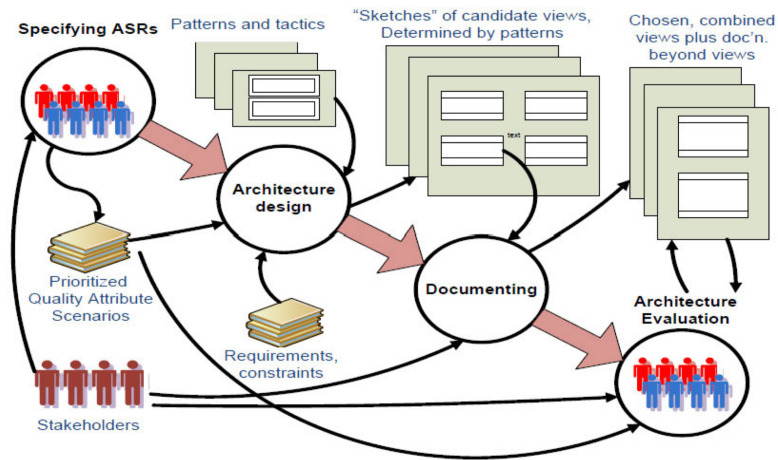
- > Where do architectures come from?

- 非功能性需求 NFRs (non-functional requirements)
 - 架构攸关需求 ASRs (architecture-significant requirements)
 - 质量需求 Quality Requirements
 - 涉众 Stakeholders
 - 组织 Organizations
 - 技术环境 Technical Environment

- 架构4+1视图 Architecture (4 + 1) Views

- 逻辑视图：功能需求中对架构重要的元素及关系
 - 过程视图：元素之间的并发与交互
 - 物理视图：如何映射到硬件
 - 开发视图：组件的内部组织联系
 - 用例场景：架构需求的特定视图

- 架构相关过程 Architecture Process



- 识别ASRs

- 输入：无

- 输出：排序后的质量属性场景

- 架构设计

- 输入：排序后的质量属性场景、需求和约束、模式和决策

- 输出：一组候选视图的草图（由模式决定）

- 架构文档化

- 输入：一组候选视图的草图（由模式决定）

- 输出：View & Beyond View

- 架构评估

- 输入：View & Beyond，排序后的质量属性场景

- 输出：View & Beyond

发现遗漏ASR则回到识别ASRs；发现不满足ASR回到架构设计

- 质量属性 Quality Attributes

- 分类

内部质量属性：和开发人员有关，使系统更容易开发维护，例如可修改性、可维护性

外部质量属性：和用户、使用者有关，例如性能、可用性

- 质量属性场景建模

- 刺激源
Source: 产生刺激的实体, 包括输入、信息等, 对当前状态的一个变化
- 刺激 Stimulus: 到达系统时需要考虑的条件
- 环境 Environment: 发生刺激时系统的状况
- 工件 Artifact: 完成需求的部分是系统的哪一部分
- 响应 Response: 刺激到来后工件开展的行为
- 响应度量 Response Measure: 对刺激的响应应以某种方法进行度量

● 质量属性

● 可用性 Availability

度量方式: ①在指定的时间间隔内, 在要求的范围内提供指定服务的概率

MTBF: 平均无故障时间

MTTR: 平均维修时间

不考虑计划内的停机时间

● 如何识别ASRs

- 从需求文档中收集 (Requirements): MoSCoW 方法和用户故事
- 通过采访涉众收集 (Workshop/Interview): 质量属性工作坊 (Quality Attribute Workshop)
- 通过了解业务目标收集 (Business Goals)
- 通过质量属性实体树 (Utility Tree) 管理: 量化描述需求, 逐渐分解细化质量属性, 直到包含量化指标

● 架构模式 Architecture Patterns

● 定义

- 上下文/背景
- 问题
- 解决方案/特点: 元素、关系、约束

● 模块化模式 Module Pattern

关注开发时各元素的关系、约束

例如分层模式 Layered Pattern, 微内核模式 Micro-Kernel Pattern

● 组件-连接器模式 Component-Connector Pattern

关注运行时各元素的关系、约束

Component 连接1对多, Connector 连接1对1

例如中继器模式 Broker Pattern, MVC 模式等

- **分配模式 Allocation Pattern**

将元素分配部署到非软件环境中

例如 Map-Reduce 充分引入并行、Multi-Tier 把一组运行在相同计算资源上的元素进行不同的组合

- **模式 vs 战术**

- 战术相比模式更简单, 使用单一的结构或机制强调单一架构要点
- 模式一般是多种设计决策的整合
- 模式和战术一起构成了软件架构的基本工具
- 战术是设计的基石, 优秀的设计升格成模式
- 大多数模式由许多不同的战术构成, 它们要么服务于功能需求, 要么用于确保各种质量属性

- **架构设计 Architecture Design**

- **设计策略**

1. 抽象 Abstraction: 关注结构本身而非实现
2. 分解 Decomposition: 分解某一个系统关注点
3. 分治 Divide & Conquer: 分别处理分解出的子模块
4. 生成与测试 Generation & Test: 根据测试路径生成测试用例
5. 迭代与细化 Iteration: 多次迭代 ADD 方法直到满足所有 ASR
6. 复用 Reuse: 重用设计中出现的可以复用的元素

- **属性驱动设计 Attribute-Driven Design**

1. 确认需求 Confirm Requirements
2. 选择一个待分解的元素 Choose an Element to Decompose
3. 识别架构攸关需求 Identify ASRs
4. 选择一种满足 ASR 的设计 Choose a Design Satisfying ASRs
 - 找出设计问题 Identify Concerns
 - 列出备选模式/决策 List Alternatives

- 从清单中选择模式/决策 Select Patterns / Tactics
- 确定模式/决策与ASR之间的关系 Determine Relations
- 记录初步的架构视图 Capture Views
- 评估并解决不一致的问题 Resolve Inconsistencies
- 5. 实例化架构元素 & 分配职责 Instantiate Elements & Allocate Responsibilities
- 6. 定义接口 Define Interface
- 7. 验证和完善需求 Verify & Refine Requirements
- 8. 重复2-7步直至满足ASRs
- 架构文档化 Documenting Architecture
 - View 部分
 - 体系结构风格和视图 Style and Views
 - 结构化视图 Structural Views
 - 模块视图 Module Views
 - 分解视图 Decomposition view
 - 使用视图 Uses view
 - 泛化视图 Generalization view
 - 分层视图 Layered view
 - 领域视图 Aspects View
 - 数据模型视图 Data model view
 - 组件-连接器视图 Component-Connector Views
 - 管道和过滤器视图 Pipe-and-filter view
 - 客户端-服务器视图 Client-server view
 - 点对点视图 Peer-to-peer view
 - 面向服务的架构 (SOA) 视图 Service-oriented architecture (SOA) view
 - 发布订阅视图 Publish-subscribe view
 - 共享数据视图 Shared-data view
 - 多层视图 Multi-tier view
 - 分配视图 Allocation Views
 - 部署视图 Deployment view

安装视图 Install view
工作分配视图 Work assignment view
其他分配视图 Other allocation views

- 质量视图 Quality Views

安全视图
性能视图
可靠性视图
通信视图
异常视图

为什么要有多种视图

- 不同视图支持不同的目标和用户，突出不同的系统元素和关系
- 不同视图将不同质量属性暴露出不同的程度

- **每一个 View 的内容**

- 主要表示：显示视图的元素和关系，以及图例
- 元素介绍：详细介绍第一部分中描述的元素、元素属性、关系属性和元素接口和行为
- 上下文图：描述系统如何与环境相关
- 可变性指南：告知视图中可能出现的变化
- 基本原理：解释设计如何映射在视图中，以及其合理性

- **如何文档化 View**

1. 构建涉众/视图表 Build Stakeholders/View Table
2. 合并视图 Combine Views
3. 确定优先级和阶段 Prioritise & Stage

- **Beyond 部分**

- 文档路线图：范围、总结、简单摘要
- 视图的文档组织方式：描述文档中视图是如何组织的
- 系统概述：从整体上描述当前架构的简要说明、业务目标（驱动因素）
- 视图之间的映射关系：描述不同视图之间的映射关系
- 系统原理：从整体上描述当前架构的设计原理
- 目录-索引、词汇表、首字母缩略词表

- 架构评估 Evaluating Architecture

- 架构分析评估方法

- 一种有助于提出正确问题的方法来发现风险、识别错误的架构决定、确保质量问题得到解决

- SAAM

- ALMA

- PASA

- ATAM

- ATAM

- > ATAM: Architecture Tradeoff Analysis Method

- 1. 阶段-0：准备和建立团队

- 参与者：评估团队领导和关键项目决策者

- 职责：根据架构设计文档生成评估计划，包括谁参加评估、如何何时何地开展评估、最后评估报告会被呈递给谁

- 输入：架构设计文档

- 输出：评估计划

- 2. 阶段-1：评估-1

- 参与者：评估团队和项目决策者

- 职责：

- 1. 评估负责人介绍 ATAM 方法 present ATAM

- 2. 项目经理或客户从业务角度介绍业务驱动因素 present business drivers

- 3. 首席架构师介绍体系结构 present architecture

- 4. 评估团队确定架构方法 identify architectural approaches

- 5. 评估团队和项目决策者生成质量属性效用树（Utility Tree） generate utility tree

- 6. 评估团队分析架构方法 analyse architectural approaches

- 输出：

1. 架构的简明介绍
2. 业务目标阐述
3. 作为场景实现的特定质量属性要求的优先列表
4. 效用树
5. 风险和无风险
6. 敏感点和权衡点

3. 阶段-2：评估-2

- 参与者：评估团队、项目决策者和项目涉众

- 职责：

1. 评估负责人介绍 ATAM 方法和之前已经取得的成果 present ATAM & results
2. 涉众头脑风暴并确定场景优先级 brainstorm & prioritize
3. 评估团队分析架构方法（类似 2.） analyse architectural approaches
4. 评估团队展示评估结果，并呈递给涉众 present results

- 输出：

1. 涉众社区的优先场景列表
2. 风险主题和业务驱动因素受到的威胁

4. 阶段-3：后续 follow-up

- 参与者：评估团队、主要涉众

- 职责：评估团队制作最终评估报告，发给主要涉众审核通过后，将报告呈递给委托评估的人。

- 输出：最终评估报告